

Bresenham's Line Drawing Algorithm

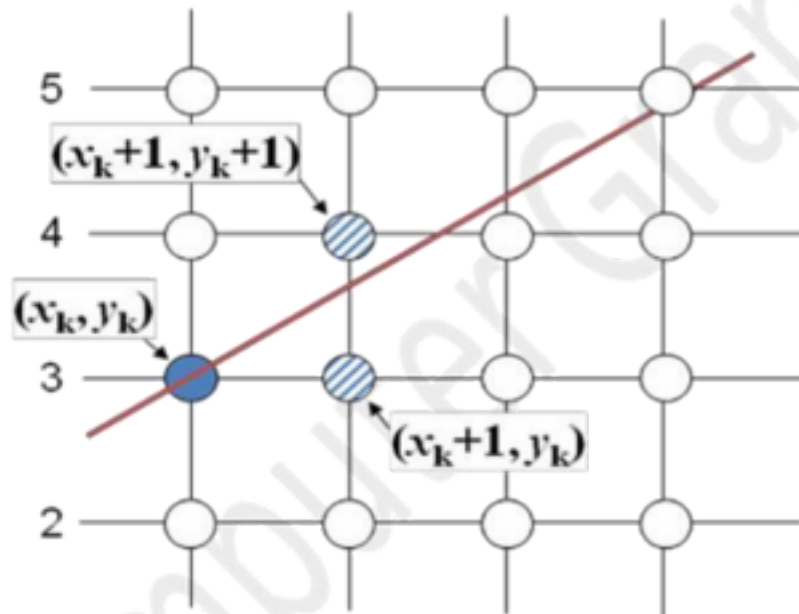
Bresenham's line algorithm is an algorithm which determines which order to form a close approximation to a straight line between two given points. Write a C program for determining pixel activation list between two given points in order to draw line segment using bresenham's Line drawing algorithm?

Aim: To implement Bresenham's line drawing algorithm for drawing a line segment between two given endpoints A (x_1, y_1) and B(x_2, y_2).

Description:

Basic Concept:

- Move across the x axis in unit intervals and at each step choose between two different y coordinates



For example, from position (2,3) we have to choose between (3,3) and (3,4). would like the point that is closer to the original line

Algorithm:

- Step 1: Input the two endpoints of the line.
- Step 2: Plot the first point (x_0, y_0).

Step 3: Calculate the constants:

Δx , Δy , $2\Delta y$, and $(2\Delta y - 2\Delta x)$.

Step 4: Calculate the initial decision parameter:

$$p_0 = 2\Delta y - \Delta x$$

Step 5: For each X_k along the line, starting with $k = 0$:

If $p_k < 0$, plot (X_{k+1}, Y_k) and set $p_{k+1} = p_k + 2\Delta y$

Otherwise, plot (x_{k+1}, y_{k+1}) and set $p_{k+1} = p_k + 2\Delta y - 2\Delta x$

Repeat step 5 until you reach the end point.

Python Implementation

```
import matplotlib.pyplot as plt
```

```
def bresenham_line(x1, y1, x2, y2):
```

```
    x, y = x1, y1
```

```
    dx = x2 - x1
```

```
    dy = y2 - y1
```

```
    dT = 2 * (dy - dx)
```

```
    dS = 2 * dy
```

```
    d = 2 * dy - dx
```

```
    x_points = [x]
```

```
    y_points = [y]
```

```
    while x < x2:
```

```

if d < 0:
    d = d + dS
else:
    d = d + dT
    y += 1
    x += 1

x_points.append(x)
y_points.append(y)

# Plot the line
plt.plot(x_points, y_points, marker='o')
plt.title("Bresenham's Line Drawing")
plt.grid(True)
plt.xlabel("X")
plt.ylabel("Y")
plt.show()

# Example Run
bresenham_line(2, 3, 10, 7)

```

Lab Exercises

Exercise 1:

Draw a line from (0, 0) to (8, 5).

- **Hint:** Check the values of `dx`, `dy`, and verify each value of `d` after every step.

Exercise 2:

Modify the program to accept user input for coordinates.

- **Hint:** Use `input()` function in Python and cast to `int`.

Exercise 3:

Implement the line drawing algorithm for lines with a **negative slope**.

- **Hint:** Modify the y-increment direction based on slope.

Exercise 4:

Print all values of `d`, `x`, and `y` in each

iteration. • **Hint:** Use `print()` inside the
loop.

Home Tasks

1. Modify the algorithm to draw lines in all 8 octants.
2. Compare output with DDA algorithm.
3. Count the number of pixels drawn for a diagonal line.
4. Extend the code to draw multiple lines using loops.